



UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO

DISCIPLINA: LABORATÓRIO DE ALGORITMOS E ESTRUTURAS DE DADOS II

DOCENTE: KENNEDY REURISON LOPES

DISCENTES: CARLOS RUBEM ALVES FERNANDES DA SILVA

FILIPPE HOLANDA DIAS

MATHEUS SANTOS MENESCAL

SISTEMA DE VERIFICAÇÃO DE CADASTRO DE USUÁRIOS

PAU DOS FERROS - RN

2026

SUMÁRIO

1 OBJETIVO	3
2 INSTRUÇÕES DE COMPILAÇÃO E EXECUÇÃO	3
3 IMPLEMENTAÇÃO E DECISÕES DE PROJETO	3
3.1 TABELA HASH	3
3.2 FILTRO DE BLOOM	4
4 EXPERIMENTOS	4
5 ANÁLISE	4
6 CONCLUSÃO	5
7 REFERÊNCIAS	5

1. OBJETIVO

Este trabalho apresenta a implementação de um sistema de cadastro de usuários que utiliza duas estruturas de dados complementares: uma tabela hash para armazenamento exato e um filtro de Bloom para consultas probabilísticas de pertencimento. O sistema foi desenvolvido em linguagem C, com identificadores de usuário de até 8 caracteres alfanuméricos. Para cada usuário cadastrado, um arquivo de texto é gerado contendo o nome deste.

2. INSTRUÇÕES DE COMPILAÇÃO E EXECUÇÃO

O projeto está estruturado em módulos separados por responsabilidade. O arquivo `hash.h` contém a implementação da tabela hash e funções auxiliares, `bloom.h` gerencia o filtro de Bloom, e `main.c` orquestra a interface com o usuário e os experimentos.

Para compilar o sistema, utiliza-se o GCC, com as flags básicas de otimização, na pasta /src:

```
gcc -o cadastro main.c -lm
```

A flag `-lm` é necessária para linkar a biblioteca matemática, utilizada nos cálculos de dimensionamento do filtro de Bloom (funções `log()` e `ceil()`).

3. IMPLEMENTAÇÃO E DECISÕES DE PROJETO

3.1 TABELA HASH

A Tabela Hash foi projetada utilizando a estratégia de Encadeamento Externo para o tratamento de colisões, onde cada posição da tabela aponta para uma lista encadeada de nós.

- Função de Hash: Implementou-se manualmente a clássica função de hash *djb2* iterativa ($\text{hash} = ((\text{hash} \ll 5) + \text{hash}) + c$), famosa por sua excelente distribuição de strings com baixo índice de colisões.
- Dimensionamento e Fator de Carga: O tamanho da tabela (`HASH_SIZE`) foi fixado no número primo 100.003. A escolha por um número primo visa reduzir agrupamentos indesejados (clustering). Com esse tamanho, no cenário de carga máxima estipulada (100.000 registros), o fator de carga (load factor, α) fica muito próximo a 1.0 ($\approx 0,99$). Com $\alpha \leq 1,0$ sob encadeamento externo, asseguramos um tempo de busca praticamente constante $O(1)$, otimizando substancialmente o desempenho.
- Identificadores: Segundo implementado nas validações (`validar_id`), os IDs de usuário foram limitados a um comprimento máximo de 11 caracteres alfanuméricos.

3.2 FILTRO DE BLOOM

O Filtro de Bloom foi implementado utilizando um vetor de bits (uint8_t) realocado dinamicamente, permitindo máxima economia de uso de memória em C.

- Dimensionamento Automático: Para não estipularmos valores arbitrários, o tamanho ótimo do vetor de bits (m) e a quantidade ideal de funções de espalhamento (k) são determinados quantitativamente na criação da estrutura em tempo de execução via `create_bloom_filter()`. Utilizamos as equações clássicas, tendo como base a taxa de falso positivo fixada no sistema em 1% ($p = 0.01$):

- $m = \left\lceil \frac{-n \ln p}{(\ln 2)^2} \right\rceil$

- $k = \left\lfloor \frac{m}{n} \ln 2 \right\rfloor$

- Funções Hash Combinadas (Double Hashing): Para evitar o custo e a lentidão de processar k funções de hash separadas, foi aplicada a otimização de Double Hashing utilizando o algoritmo genérico FNV-1a de 64-bits com duas sementes (FNV_SEMENTE_1 e FNV_SEMENTE_2). O índice procurado é ativado pelas k iterações simuladas combinando as duas bases, o que mantém alta precisão sem comprometer a CPU.

4 EXPERIMENTOS

O arquivo `main.c` roda uma bateria de perfis quantitativos (1.000, 10.000 e 100.000 registros) em memória e contabiliza os desempenhos com `clock_t` da biblioteca `<time.h>`. Para cada rodada:

1. Os IDs randômicos são gerados e inseridos nas duas estruturas.
2. Mede-se o tempo real da busca por chaves já existentes nas bases.
3. Executa-se a busca por chaves não inseridas, para averiguar quando o Filtro de Bloom informa a existência de forma incorreta. Nesse caso, a chave é procurada na Hash para constatar o Falso Positivo.

5 ANÁLISE

Respondendo às questões analíticas propostas nos documentos do projeto:

1. O Filtro de Bloom reduziu consultas na Tabela Hash?

Sim. O filtro atua como uma barreira inicial. Ao pesquisar um ID que ainda não foi cadastrado, o Filtro de Bloom aponta que ele "definitivamente não existe" na imensa

maioria dos casos (próximo de 99%). Isso evita operações dispendiosas, como a busca iterativa nas listas encadeadas (resolução das colisões) dentro da Hash Table.

2. Como o tamanho do vetor impactou os resultados?

Graças ao cálculo dimensionamento do vetor implementado em bloom.c, os bits necessários escalam em proporção aos elementos (por exemplo, exigirá muito menos memória ao rodar testes com 1.000 elementos e escalará adequadamente para suportar 100.000 registros mantendo o fator de ocupação coerente). Isso garantiu que os resultados se conservassem constantes, mantendo a taxa e a eficiência uniformes independentemente do cenário sem sobrecarregar a memória do processo desnecessariamente.

3. O número de funções hash alterou a precisão?

Sim, a precisão foi mantida estável devido à alocação calculada de k . Se definíssemos as funções de forma aleatória sem levar em conta o α (fator de ocupação de bits), o vetor ficaria extremamente saturado precocemente. Ao utilizar o número ideal (a função que rege o erro de Bloom), obtivemos a distribuição simétrica no Double Hashing, travando a margem de erros (Falsos Positivos) ao índice almejado de exatos 1%.

6 CONCLUSÃO

A conjunção desenvolvida da Tabela Hash para consistência dos registros aliados com a leveza do Filtro de Bloom cumpriu perfeitamente os requisitos apresentados no escopo inicial. Enquanto a Hash garante estabilidade nas chaves recuperadas ($O(1)$ assegurado com as propriedades do fator de carga e encadeamento), a heurística do Bloom blindou a aplicação contra o custo computacional de procuras inválidas. Este formato demonstrou-se como o balanço ideal entre o consumo restrito de memória e as demandas severas de desempenho exigidas em bases de cadastros gigantescas.

7 REFERÊNCIAS

KIRSCH, Adam; MITZENMACHER, Michael. Less hashing, same performance: Building a better Bloom filter. 2008. **Random Structures & Algorithms**.

NOLL, Landon Curt. **FNV Hash**. Disponível em: <http://www.isthe.com/chongo/tech/comp/fnv/>. Acesso em: 30 jun. 2026.

KNUTH, Donald Ervin. **The Art of Computer Programming, v. 3: Sorting and Searching**. 2. ed. Addison-Wesley, 1998.